

A Practical Introduction to Diffusion Models

Zhihao Li

Homepage: <https://lzhms.github.io/>

Email: zhihaoli25@stu.pku.edu.cn

1	Lecture 1: Introduction to diffusion models	2
1.1	Applicaitons of diffusion models	2
1.2	Diffusion Model	5
2	References	11

1 Lecture 1: Introduction to diffusion models

1.1 Applications of diffusion models

1.1.1 Text-to-image generation



Figure 1: Image credit: [SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis](#)

1.1.2 Sketch-to-Image: coarse-grained control

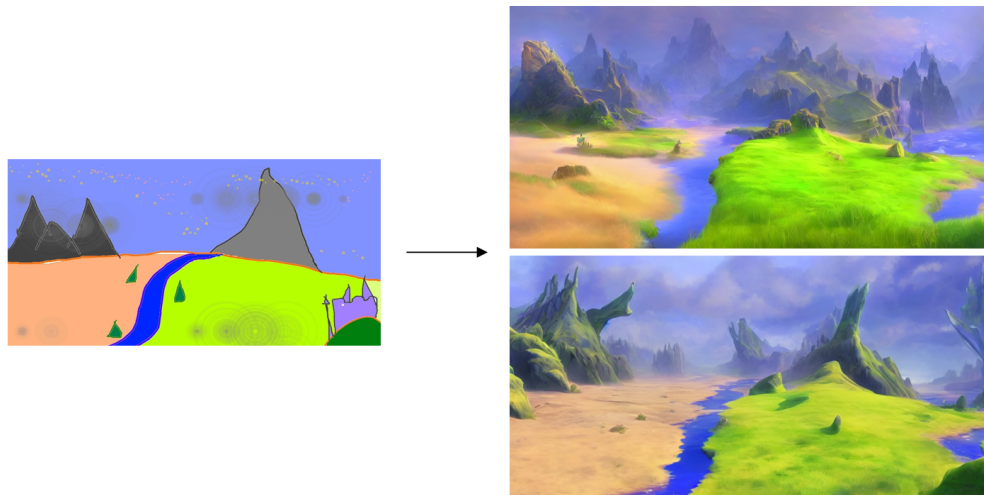


Figure 2: Image credit: [Stable Diffusion](#)

1.1.3 ControlNet: fine-grained control

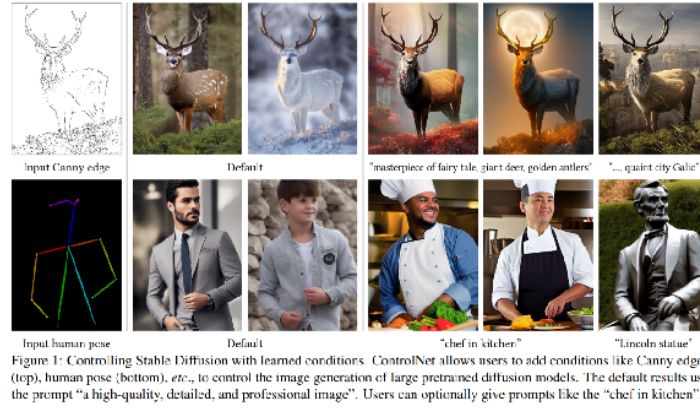


Figure 3: Image credit: [Adding Conditional Control to Text-to-Image Diffusion Models](#)

1.1.4 Image editing and composition



Figure 4: Image credit: [DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation](#)

1.1.5 Diffusion “prior” for image restoration

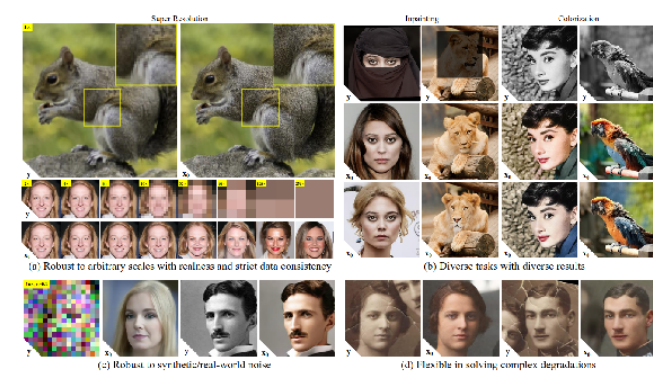


Figure 5: Image credit: [Zero-Shot Image Restoration Using Denoising Diffusion Null-Space Model](#)

1.1.6 Generating visual illusions

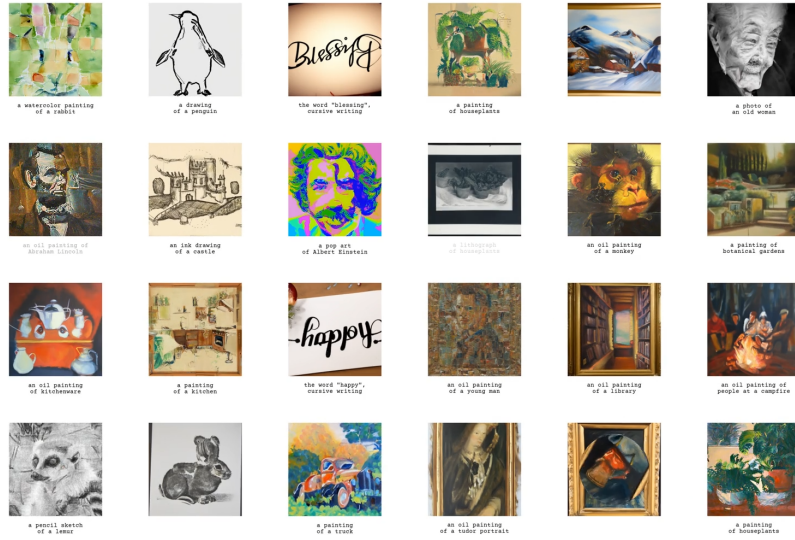


Figure 6: Image credit: [Visual Anagrams: Generating Multi-View Optical Illusions with Diffusion Models](#)

1.1.7 Text-to-audio generation

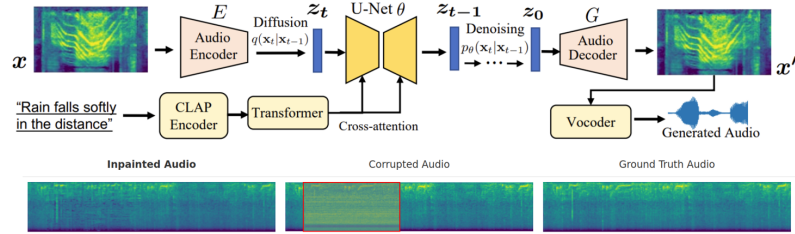


Figure 7: Image credit: [Make-An-Audio: Text-To-Audio Generation with Prompt-Enhanced Diffusion Models](#)

1.1.8 Video generation



Figure 8: Video generation

1.2 Diffusion Model

扩散模型的名字来源于扩散过程（布朗运动，Brownian motion），如图9所示，从零点开始随着时间演化，形成了一条随机的轨迹。

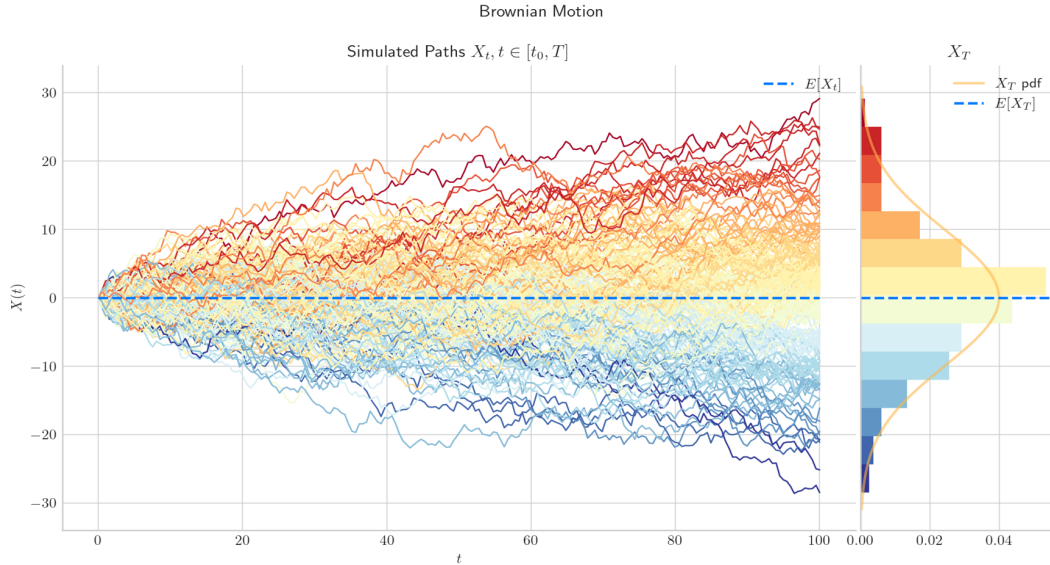


Figure 9: 布朗运动 (Brownian motion)

在概率论中，布朗运动指的是一个连续时间、连续路径的随机过程 $\{X(t), t \geq 0\}$ 满足：

1. $X(0) = 0$ ，即布朗运动从零点开始。
2. 对于任意的 $0 \leq s < t$ ，增量 $X(t) - X(s)$ 服从均值为 0、方差为 $t - s$ 的正态分布，即 $X(t) - X(s) \sim \mathcal{N}(0, t - s)$ 。
3. 布朗运动具有独立增量性质，即对于任意的 $0 \leq t_1 < t_2 < \dots < t_n$ ，增量 $X(t_2) - X(t_1), X(t_3) - X(t_2), \dots, X(t_n) - X(t_{n-1})$ 相互独立。
4. 布朗运动的路径是连续的，但几乎处处不可微。
5. 布朗运动具有平稳增量性质，即增量的分布仅依赖于时间间隔 $t - s$ ，而与具体的时间点 s 和 t 无关。

如果对于高维度数据，例如图像，扩散模型在每一个像素上添加噪声，forward 过程就是对数据不断添加噪声直至变成纯噪声；reverse 过程就是学习从纯噪声逐步去噪还原出数据的过程。

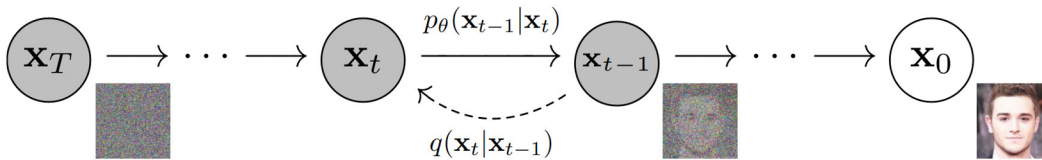


Figure 10: Denoising Diffusion Probabilistic Models

Training Diffusion Models 去噪扩散模型根据给定的噪声水平 (Noise Level) $\sigma > 0$ 和噪声输入 (Noise Input) $x_\sigma \in \mathbb{R}^n$ 估计一个噪声向量 (Noise Vector) $\epsilon \in \mathbb{R}^n$ 使得对于数据流形 (Data Manifold) K 中的某个 x_0 , 满足以下关系:

$$x_\sigma = x_0 + \sigma \epsilon. \quad (1)$$

其中, 噪声向量可以看作是是一个方向, 选择正确的噪声水平和方向, 添加到干净的数据上就会得到更加噪声的数据。

扩散模型的任务就是预测噪声是以何种方向添加的, 即预测噪声向量 (噪声水平是预先设定的)。给定 Noisy Input 以及 Noise Level, 模型试图从噪声输入中预测 Noise Direction.

一个 Denoiser $\epsilon_\theta : \mathbb{R}^n \times \mathbb{R}_+ \rightarrow \mathbb{R}^n$ 通过最小化以下损失函数进行训练:

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0 \sim p_{data}, \epsilon \sim \mathcal{N}(0, I_n), \sigma \sim p(\sigma)} [\|\epsilon - \epsilon_\theta(x_0 + \sigma \epsilon, \sigma)\|_2^2]. \quad (2)$$

其中, x_0 是从数据分布 p_{data} 中采样的干净数据, ϵ 是从标准正态分布 $\mathcal{N}(0, I_n)$ 中采样的噪声向量, σ 是从噪声水平分布 $p(\sigma)$ 中采样的噪声水平。

Training Noise Schedules 噪声水平 σ 实际上可以认为是添加噪声的步长, 其可大可小, 选择范围从 $\sigma_{\min} = 0.01$ 到 $\sigma_{\max} = 100$ 。相比于数据幅度, 噪声水平可以选择比数据幅度较小或较大的值。

噪声水平实际上是从 Noise Schedule 中均匀采样, Noise Schedule 是包含例如 100-1000 个离散噪声值的集合。

$$\sigma \sim \{\sigma_1, \sigma_2, \dots, \sigma_{1000}\}. \quad (3)$$

由于噪声水平可以取值很大, 一般遵循 log-scale 的缩放 $\lambda = \log \sigma$ 。

Toy Model 为了更好地理解扩散模型训练流程, 可以考虑一个简单的二维 toy model, 如图11所示。假设数据分布是一个环形分布, 通过拼接加噪的数据以及设定的 Noise-level Embedding 作为模型的输入, 训练一个简单的 MLP 来预测噪声方向。

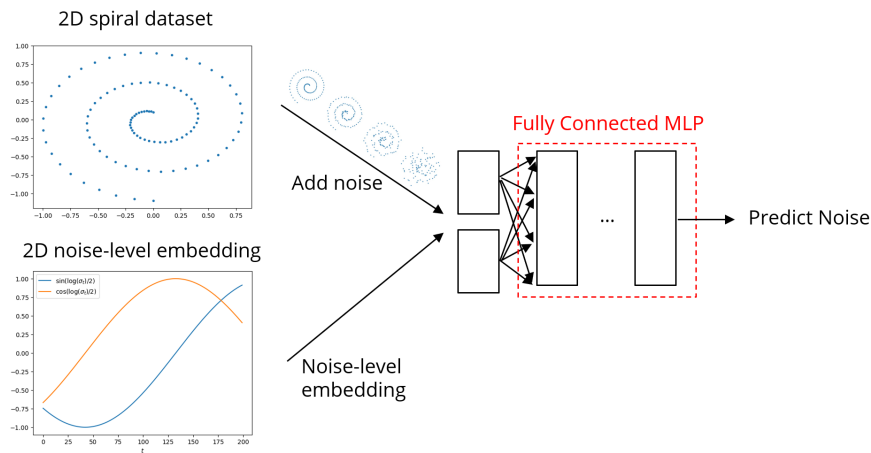


Figure 11: A toy model for understanding diffusion models

简单的实现框架如下所示:

```

1  import torch
2  import torch.nn as nn
3  from itertools import tee
4
5  # The 'pairwise' function is not a standard Python built-in before 3.10.
6  # Here is a common implementation from the itertools recipes.
7  def pairwise(iterable):
8      "s -> (s0,s1), (s1,s2), (s2, s3), ..."
9      a, b = tee(iterable)
10     next(b, None)
11     return zip(a, b)
12
13 def get_sigma_embeds(sigma):
14     sigma = sigma.unsqueeze(1)
15     return torch.cat([torch.sin(torch.log(sigma)/2),
16                       torch.cos(torch.log(sigma)/2)], dim=1)
17
18 class TimeInputMLP(nn.Module):
19     def __init__(self, dim, hidden_dims):
20         super().__init__()
21         layers = []
22         # The input dimension for the first layer is dim + 2 (for x and sigma_embeds)
23         all_dims = (dim + 2,) + hidden_dims
24         for in_dim, out_dim in pairwise(all_dims):
25             layers.extend([nn.Linear(in_dim, out_dim), nn.GELU()])
26
27         # Final layer maps from the last hidden dimension back to the original dimension
28         layers.append(nn.Linear(hidden_dims[-1], dim))
29
30         self.net = nn.Sequential(*layers)
31         self.input_dims = dim # Storing the original input dimension
32
33     def rand_input(self, batchsize):
34         # NOTE: The original image has `torch.randn(batchsize,) + self.input_dims`.
35         # This is likely a typo and would cause an error.
36         # The probable intention was to create a random tensor of shape (batchsize, dim).
37         # return torch.randn(batchsize, ) + self.input_dims # Original from image (likely incorrect)
38         return torch.randn(batchsize, self.input_dims) # Corrected version
39
40     def forward(self, x, sigma):
41         sigma_embeds = get_sigma_embeds(sigma) # shape: b x 2
42         nn_input = torch.cat([x, sigma_embeds], dim=1) # shape: b x (dim + 2)

```

```

43     return self.net(nn_input)
44
45 # Example of creating the model instance
46 model = TimeInputMLP(dim=2, hidden_dims=(16, 128, 128, 128, 128, 16))
47
48 # You can print the model to see its structure
49 print(model)

```

Commonly Used Model Architectures

Convolutional U-Nets Analyzing and Improving the Training Dynamics of Diffusion Models

第一种常用的扩散模型结构是基于卷积层的 U-Nets 网络，通过设置编码器执行下采样，解码器执行上采样，并在每个对应的编码器和解码器层之间添加跳跃连接（Skip Connection），以保留空间信息，如图12所示。

其中，将噪声水平作为条件输入，指导扩散模型的去噪过程。

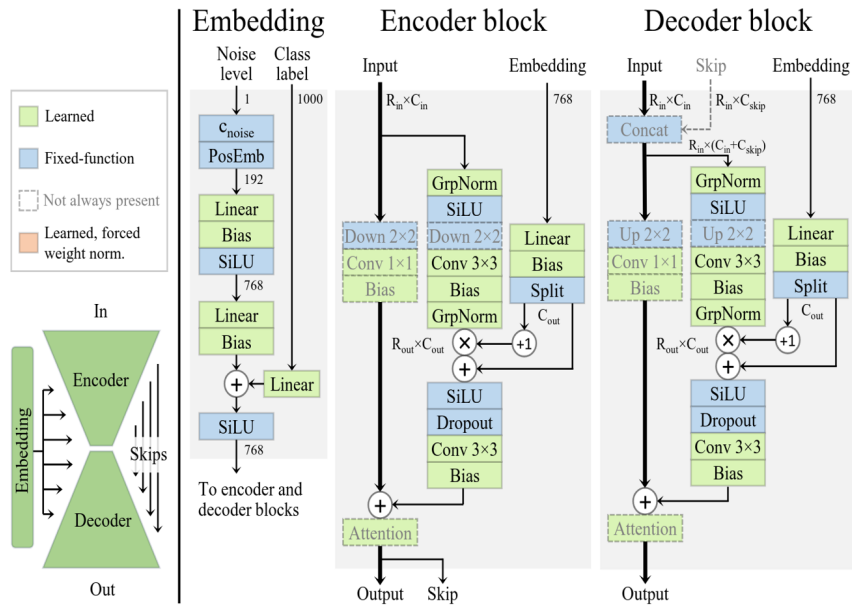


Figure 12: Convolutional U-Nets: Analyzing and Improving the Training Dynamics of Diffusion Models

Patch-wise Transformers Scalable Diffusion Models with Transformers

第二种常用的模型结构是基于最近的 Transformer，首先将图像分成多个 Patch 作为模型的输入。堆叠若干个 Transformer Block 条件化噪声输入（Noised Latent）和噪声水平（Noise Level Embedding），预测在噪声图像中添加噪声方向，如图13所示。

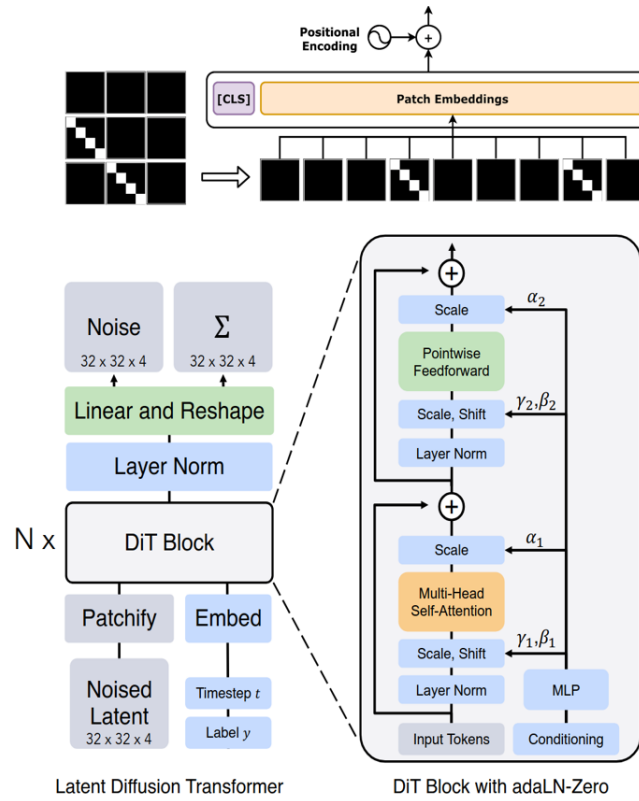


Figure 13: Patch-wise Transformers: Scalable Diffusion Models with Transformers

Training the Toy Model 利用上面定义的 Toy Model 设计训练过程:

```

1  def training_loop(loader: DataLoader,
2      model: nn.Module,
3      schedule: 'Schedule', # Using quotes as Schedule is a placeholder
4      epochs: int = 10000):
5
6      optimizer = torch.optim.Adam(model.parameters())
7
8      for _ in range(epochs):
9          for x0 in loader:
10             optimizer.zero_grad()
11
12             # Generate a noise level (sigma) and noise vector (eps)
13             sigma, eps = generate_train_sample(x0, schedule)
14
15             # The model predicts the noise vector from the noisy input
16             eps_hat = model(x0 + sigma * eps, sigma)
17

```

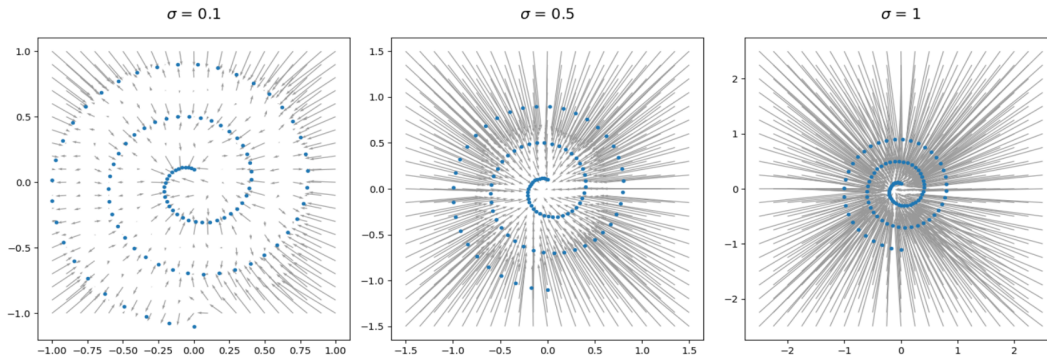
```

18 # Calculate the mean squared error between the predicted noise and the actual noise
19 loss = nn.MSELoss()(eps_hat, eps)
20
21 # NOTE: The line below, `optimizer.backward(loss)`, is unconventional.
22 # The standard PyTorch practice is to call `loss.backward()`.
23 # The code is transcribed exactly as it appears in the image.
24 # loss.backward() # This is the typical way to compute gradients.
25 optimizer.backward(loss) # As written in the image.
26
27 optimizer.step()

```

学习的 Denoiser $\epsilon_\theta(x, \sigma)$ 通过参数化噪声水平 σ 可以被可视化为一个向量场，如下图所示。

对于不同的 x 以及噪声水平 σ ，绘制 $x - \sigma\epsilon_\theta(x, \sigma)$ 的向量场，当噪声水平较大时，模型倾向于预测数据的平均值，可能因为噪声水平占据了主导部分；当噪声水平较小时，模型倾向于预测数据点本身。



Plot of predicted $\hat{x}_0 = x - \sigma\epsilon_\theta(x, \sigma)$ for different x and σ

What did the model learn? 理想的 Denoiser 是关于数据分布 $\mathcal{K}$ 和噪声水平 σ 的训练损失的最小化器：

$$\epsilon^* := \arg \min_{\epsilon_\theta} \mathbb{E}_{x_0, \sigma, \epsilon} [\|\epsilon - \epsilon_\theta(x_0 + \sigma\epsilon, \sigma)\|_2^2]. \quad (4)$$

对于有限数据分布 $\mathcal{K}$ ，存在闭包解：

$$\epsilon^*(x_\sigma, \sigma) = \frac{\sum_{x_0 \in \mathcal{K}} (x_\sigma - x_0) \exp\left(-\frac{\|x_\sigma - x_0\|^2}{2\sigma^2}\right)}{\sigma \sum_{x_0 \in \mathcal{K}} \exp\left(-\frac{\|x_\sigma - x_0\|^2}{2\sigma^2}\right)}. \quad (5)$$

2 References

- [1] Chenyang Yuan et al. *MIT 6.S183: A Practical Introduction to Diffusion Models*. 2025. URL: <https://www.practical-diffusion.org/> (cit. on p. 1).